第四届eBPF开发者大会

The 4th eBPF Developer Conference (eBPFDC 2026)

陕西 · 西安　　2026年4月18日

https://www.ebpftravel.com

bpftime for GPU

将 eBPF 带入 GPU Kernel 内部 · PTX 级插桩 · Per-Thread 可观测性

陕西 · 西安　　2026年4月18日

目录

01 为什么需要 GPU eBPF
传统工具的能力边界与 bpftime 的定位

02 bpftime GPU 架构
nv_attach_impl · PTX Pass · GPU Maps · Helpers

03 核心技术：PTX 插桩流水线
eBPF C → LLVM IR → PTX → cubin → 执行

04 GPU 专用 Maps 与 Helpers
Ring Buffer · globaltimer · threadIdx · 零拷贝回传

05 性能基准
uprobe 13-16x | GPU 探针 vs NVBit 2-14x

06 支持的探针类型与示例
kprobe / kretprobe / memcapture · 20+ 示例

07 工具对比
bpftime vs Nsight vs CUPTI vs NVBit

08 路线图
实时 UI · bpftrace 集成

① 为什么需要 GPU eBPF

传统工具的能力边界 vs bpftime GPU

传统工具能做到

- Kernel 整体执行耗时
- SM 级别 occupancy
- L1/L2 cache hit rate
- warp scheduler 利用率
- CUDA API 调用序列（cudaMemcpy 等）
- Grid / Block 配置记录

Nsight Systems / Nsight Compute / CUPTI

⚠ *以上均为kernel 外部、warp/SM 级别的聚合观测*

→

bpftime GPU 能做到 ★

- **Per-thread 退出时间戳**
  纳秒精度，每个线程独立
- **Kernel 内部任意位置插桩**
  在任意 ret 指令前注入探针
- **可编程 eBPF C 探针**
  自定义采集逻辑，无预定义限制
- **选择性插桩**
  只 hook 目标 kernel，其他零开销
- **GPU→Host 零拷贝回传**
  UVA 映射，无需 cudaMemcpy
- **内存访问模式追踪**
  ptxpass_memcapture load/store

这是目前 eBPF 生态中独一无二的能力

② bpftime GPU 架构

bpftime GPU 架构：nv_attach_impl

1 CUDA API 拦截
Frida-gum hook
__cudaRegisterFatBinary()

→

2 PTX 提取
cuobjdump
--extract-ptx

→

3 PTX Pass 变换
kprobe_entry
kretprobe
memcapture

→

4 eBPF → PTX
LLVM JIT
bytecode → PTX

→

5 Register Guard
保存/恢复寄存器
不破坏原始状态

→

6 PTX → cubin
nvPTXCompiler
sm_75/80/90

→

7 替换 GPU Module
应用无感知
带探针执行

动态挂载（Dynamic Attach）

首次挂载耗时

~100ms

PTX 改写 + 重编译，后续 launch 无额外开销

无需重启应用
运行中的 GPU kernel 可即时动态注入探针

多 GPU 支持
同时对多卡 GPU 插样，全局统一观测

实现原理
Frida-gum 拦截 CUDA API → cuobjdump 提取 PTX → PTX Pass 改写 → 重新加载 cubin

应用无感知
无需修改源码，不需 root 权限，带探针的 cubin 透明替换原始 module

③ PTX 插桩流水线

核心技术：PTX 插桩流水线

编译链路

eBPF C 源码
clang -target bpf
42 行 kernelretsnoop.bpf.c

↓

eBPF 字节码
64-bit RISC 指令集
11 个寄存器

↓

LLVM IR
bpftime LLVM JIT
compile_ebpf_to_ptx_from_words()

↓

PTX 汇编
注册 GPU helper 映射
生成 add_register_guard()

↓

sm_75 cubin
nvPTXCompiler 编译
替换原始 GPU module

PTX Pass 改写

kretprobe：在每个 ret 指令前注入探针调用
kprobe：在 kernel 入口插入计数器更新
memcapture：将 load/store 替换为记录指令

Register Guard

探针前后自动生成寄存器保存/恢复代码
确保探针执行完后原始 kernel 状态完全不变
add_register_guard() 在 LLVM PTX 生成阶段插入

跳过重编译开销

首次 launch 需 PTX 改写 + nvPTXCompiler
后续 launch 直接执行已加载的 cubin，无额外开销

④ GPU 专用 Maps 与 Helpers

GPU Map 类型（基于 1500+）

1501 **GPU_HASH_MAP** 显存哈希表

1502 **PERGPUTD_ARRAY_MAP** 显存 per-thread 数组

1503 **GPU_ARRAY_MAP** 显存共享数组

1504 **GPU_KERNEL_SHARED_ARRAY_MAP**
Kernel 直接访问的共享显存

1512 **PERGPUTD_ARRAY_HOST_MAP**
Host 内存 per-thread，Tegra 平台

1513 **GPU_ARRAY_HOST_MAP**
Host 内存共享，Tegra 平台

1527 **GPU_RINGBUF_MAP** ★
Per-thread 无锁环形缓冲，UVA 零拷贝回传

GPU Helper 函数（ID 501–511）

501 **ebpf_puts()** GPU 打印到 host

502 **bpf_get_globaltimer()** 纳秒时钟 ★

503 **bpf_get_block_idx()** blockIdx (x,y,z)

504 **bpf_get_block_dim()** blockDim (x,y,z)

505 **bpf_get_thread_idx()** threadIdx (x,y,z) ★

506 **bpf_gpu_membar()** membar.sys 内联 PTX

507 **bpf_gpu_exit()** 终止当前线程

508 **bpf_get_grid_dim()** gridDim (x,y,z)

509 **bpf_get_sm_id()** SM 编号

510 **bpf_get_warp_id()** SM 内 Warp ID

511 **bpf_get_lane_id()** Lane ID 0–31

⑤ 性能基准测试

性能基准测试

CPU 侧：uprobe / uretprobe 延迟

操作	内核 eBPF	bpftime	加速比
uprobe	2,561 ns	190 ns	**13.5x ★**
uretprobe	3,019 ns	187 ns	**16.1x ★**
uprobe+uretprobe	3,119 ns	192 ns	**16.3x ★**
用户态内存读取	22,506-30,934 ns	1,491-1,862 ns	**14-21x ★**

uprobe 13-16x
消除 context switch 开销

GPU 探针 vs NVBit
RTX 5090 快 6.8x

更新 GPU 架构
bpftime 优势越来越大

GPU 侧：10,000 次 vector addition | 探针开销对比（bpftime vs NVBit）

GPU 设备	无探针基线	bpftime	NVBit	bpftime 开销	NVBit 开销
NVIDIA P40	51.8 µs	81.1 µs	174.4 µs	**1.56x ✓**	3.37x
NVIDIA RTX 5090	4.1 µs	8.2 µs	55.8 µs	**2.0x ✓**	13.6x

⑥ 探针类型

探针类型与示例生态

GPU kprobe

在 GPU kernel 入口注入探针

每个 block 触发一次（CUDA grid 粒度）

适合采集：入口时间戳、调用计数、blockIdx

ATTACH_CUDA_PROBE (type 8)

GPU kretprobe

在 GPU kernel 每个 ret 指令前注入探针

每个线程触发一次（最细粒度）

适合采集：退出时间戳、per-thread 执行状态

ATTACH_CUDA_RETPROBE (type 9)

Memory Capture

对 kernel 中的 load/store 指令插样

每次内存访问均触发

适合采集：内存访问模式分析

ptxpass_memcapture

仓库内 20+ GPU 示例（example/gpu/）

cuda-counter | kernelretsnoop | threadhist | mem_trace | pytorch-test | llama-cpp-test | faiss-test

cutlass | cudagraph | launchlate | gpu_shared_map | rocm-counter | threadhist-gpu-kernel-shared-map | ...

⑦ 工具对比

工具对比

能力维度	bpftime	Nsight Systems	CUPTI	NVBit
零代码修改	✓	✓	✗	✓
无需 root	✓	✗	✓	✓
插桩层级	PTX	API 层	API 层	SASS
可编程探针 ★	✓	✗	✗	✗
Per-thread 数据 ★	✓	✗	✗	✗
Kernel 内部插桩 ★	✓	✗	✗	✓
内存访问追踪 ★	✓	✗	✗	✗
硬件性能计数器	✗	✓	✓	✗
开源	✓Apache 2.0	✗闭源	✗闭源	✓NVIDIA

★ = bpftime 独有能力 | NVBit 插桩 SASS（原生机器码），bpftime 插桩 PTX（虚拟指令集，天然跨 GPU 架构）

⑧ 路线图 · 参考资料

路线图

最近更新

✓ 大幅度优化 attach 开销
✓ 支持 attach 到正在运行的进程
✓ 支持多 GPU
✓ 和 driver 协同工作

路线图

扩展 memcapture
→ 寄存器值 + shared memory 模式
AMD ROCm/HIP 后端
GCN/RDNA 指令集支持
实时流式 profiling
→ 类 Nsight 的在线可视化
bpftrace 集成
→ GPU 版 one-liner 采集语法

“Extending eBPF to GPU Device and Driver Contexts” Linux Plumbers Conference 2025
“gpu_ext: Extensible OS Policies for GPUs via eBPF” https://arxiv.org/abs/2512.12615
"Extending eBPF Programmability and Observability to GPUs" HCDS '25

bpftime for GPU

将 eBPF 的可编程观测能力延伸至 GPU 内部

- PTX 级插桩 → kernel 内任意 ret 注入探针
- Per-thread 纳秒时间戳 → 20,480 线程级事件
- uprobe 13-16x 快于内核 eBPF，GPU 探针快于 NVBit 2-14x
- GPU Ring Buffer + UVA → 零拷贝数据回传
- 兼容标准 eBPF 工具链（clang / libbpf / bpftrace）

github.com/eunomia-bpf/bpftime

陕西 · 西安　　2026年4月18日